

Potentiometer-to-SCOMP Interface: ADC Peripheral Design

Kien Le

Submitted

April 28, 2026

Introduction

This document describes the ADC peripheral created for the SCOMP system, enabling programmers to smoothly incorporate potentiometers through eight analog input channels. The peripheral provides a easy-to-use interface that simplifies potentiometer and ADC communication for the user, including error indications (0xF000) to flag invalid configuration or unresponsive channels.

Our approach mainly prioritize accessing the 8 different channels as well as being able to implement the read/write functionality via simple memory-mapped writes to addresses 0xC0-0xC7. While we were able to develop the peripheral's core features, it was ultimately unsuccessful due to communication mismatches between the peripheral and the SCOMP system. Time constraints and conflicting schedules limited our ability to debug and resolve these integration issues before the demonstration.

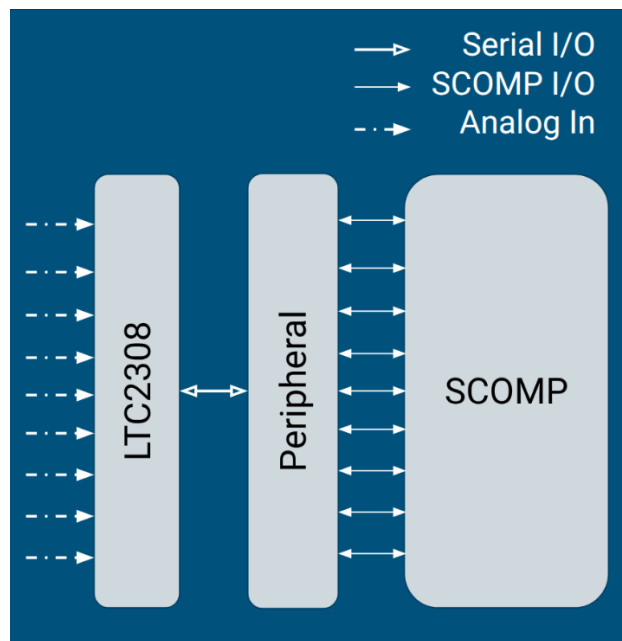


Figure 1. Data flow: from analog input to SCOMP-readable digital values

Device Functionality

The peripheral operates to simplify the conversion pathway from the potentiometer device to ADC to SCOMP programmer. By abstracting low-level ADC timing and protocol details, the hardware peripheral was created to provide an interface that allowed easy and accessible data reading and configuration for the user to create

programs using the desired device.

Reading from the Device (IN)

With the current peripheral layout, the SCOMP programmer can read from any of the eight desired channels by referring to any of the addresses from 0xC0 - 0xC7. From there, when the user wants to utilize or display the data from the selected channel, they simply have to use the IN instruction to get the channel data into the accumulator (AC).

Writing to the Device (OUT)

If the user wanted to configure the peripheral for the unipolar or the bipolar, or keep the current setup for the channel. When the user OUTs a negative value from the AC out to the channel, the peripheral will be set to bipolar. A positive value will set the channel to unipolar, and a zero will indicate the same configuration. Once the channel is formatted with the desired setup, the previously stored data is invalidated to prevent error data.

Address	Name	I/O	Data															
			15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0xC0	CH0	READ	ERR_CH0								DATA_CH0							
0xC1	CH1		ERR_CH1								DATA_CH1							
0xC2	CH2		ERR_CH2								DATA_CH2							
0xC3	CH3		ERR_CH3								DATA_CH3							
0xC4	CH4		ERR_CH4								DATA_CH4							
0xC5	CH5		ERR_CH5								DATA_CH5							
0xC6	CH6		ERR_CH6								DATA_CH6							
0xC7	CH7		ERR_CH7								DATA_CH7							

Figure 2. IO Data breakdown demonstrating 0xF000 flag for invalid configuration or unresponsive channels

Design Decisions and Implementation

During the design process, our group went over the basis of what interface we wanted to create a peripheral for, which in this case were potentiometers. We broke down the ADC and its functionalities to determine 8 channels that the ADC reads analog data from before converting to digital.

With this in mind, we designed the peripheral to continuously poll through these channels, updating each one with the most recent data from the device. This process loops within the system as a round robin approach.

We wanted to utilize all 8 channels and not just hardcode it to the two designated channels (0 and 2) in lab, so this setup addresses situations where a user might want to access the other channels for data.

Significant Modifications Made During Design

Initially we wanted only the active channel approach, where the channel that the user was referencing would be the only channel with updated data; however, due to time constraints with testing, we deferred to round robin polling instead as it continuously loops through the 8 channels in the background ensuring that the data would be updated based on the user configuration. While there are tradeoffs of timing for this method, polling was easier to implement and test than active channel switching.

Conclusions

While the peripheral implementation failed to execute the demonstration program, valuable lessons were gained from the planning, design, and prototype development phases. Developing the peripheral was a valuable experience in learning how the SPI interface worked in the communication pathway from analog to digital data. One key recommendation for the implementation would be to include embedded timing detection to detect system hangs when a timeout occurs with the ADC.

Additionally, for future applications of this peripheral, engineers integrating this peripheral should implement software debouncing for potentiometer noise. In retrospect, instead of the polling system, I would want to incorporate the active channel as it would better suit applications using only 1-2 channels rather than parsing through all eight when seven are inactive.